

# Programiranje za web

## Predavanje 3: Autentikacija i sigurnost

doc. dr. sc. Tomislav Petković   mag. ing. Josip Torić

22.10.2026.



**Sveučilište u Zadru**  
Universitas Studiorum  
Jadertina | 1396 | 2002 |

# Plan predavanja

- 1 Identitet u protokolu bez pamćenja
- 2 Lozinke: zašto se ne pohranjuju i kako se provjeravaju
- 3 Implementacija tokena: sadržaj, potpis, vijek, dvije vrste
- 4 Prijava i provjera pri svakom zahtjevu
- 5 Što token ne rješava

Na kraju predavanja korisnik se može prijaviti, dobiti token i njime doći do zaštićene rute.

# Ishodi učenja

Nakon ovog predavanja student može:

- razgraničiti autentikaciju od autorizacije
- objasniti čemu služi salt i zašto se za lozinke bira spor algoritam
- opisati sadržaj tokena i reći što potpis jamči, a što ne
- obrazložiti postojanje dviju vrsta tokena i razliku u njihovu vijeku
- provesti provjeru identiteta pri svakom zahtjevu i imenovati što ona ne pokriva

# Domena primjera

Sustav prijava klubova na powerlifting natjecanje. Iz prethodnog predavanja u bazi postoje tablice `clubs` i `users`, a početnim punjenjem u njima su organizator, jedan klub i račun toga kluba.

Korisnik ima adresu e-pošte za prijavu, hash lozinke, ulogu i, ako pripada klubu, strani ključ prema njemu. Danas se ti zapisi prvi put upotrebljavaju.

---

# Identitet u protokolu bez pamćenja

Zašto zahtjev ne zna tko ga šalje, zaštita prijenosa, autentikacija i autorizacija, sjednica i token.

# HTTP ne pamti pošiljatelja

Svaki HTTP zahtjev dolazi sam za sebe. Poslužitelj iz njega vidi metodu, putanju, zaglavlja i tijelo, i ništa više:

```
GET /clubs/12/lifters HTTP/1.1  
Host: api.primjer.hr
```

U tom zapisu nema ničega što bi kazalo tko ga šalje. Prethodni zahtjev istog korisnika nije ostavio trag, a veza kojom je stigao ne nosi identitet.

Iz toga slijedi da identitet mora putovati **u samom zahtjevu**, i to u svakom zahtjevu posebno.

# HTTP i HTTPS

Zahtjev i odgovor putuju kao običan tekst. Svatko tko je na putu, od vlasnika pristupne točke do davatelja pristupa, čita ih u cijelosti i može ih izmijeniti.

HTTPS je isti HTTP unutar šifriranog sloja TLS. Taj sloj daje troje:

- sadržaj zahtjeva i odgovora nije čitljiv onima na putu
- izmjena u prijenosu se otkriva
- certifikat potvrđuje da je poslužitelj doista tražena domena

Za sve što slijedi to je preduvjet, a ne dodatak: bez njega lozinka pri prijavi i vjerodajnica u svakom sljedećem zahtjevu putuju čitljive. Norma koja opisuje prilaganje tokena zahtjevu (RFC 6750) zato izrijeком traži TLS.

Iznimka je razvoj na vlastitom računalu: preglednici localhost smatraju sigurnim izvorom, pa ondje običan HTTP ne predstavlja rizik.

# Autentikacija i autorizacija

Prije nego zahtjev bude obrađen, odgovaraju se dva pitanja. Prvo se tiče pošiljatelja, drugo onoga što traži:

| Pojam         | Pitanje             | Ishod ako ne prođe |
|---------------|---------------------|--------------------|
| autentikacija | tko šalje zahtjev   | 401 Unauthorized   |
| autorizacija  | smije li to učiniti | 403 Forbidden      |

Redoslijed je nužan: dok se ne zna tko je pošiljatelj, nema se o čijim pravima odlučivati. Ovo predavanje obrađuje samo prvo pitanje.



# Kako se korisnik autenticira

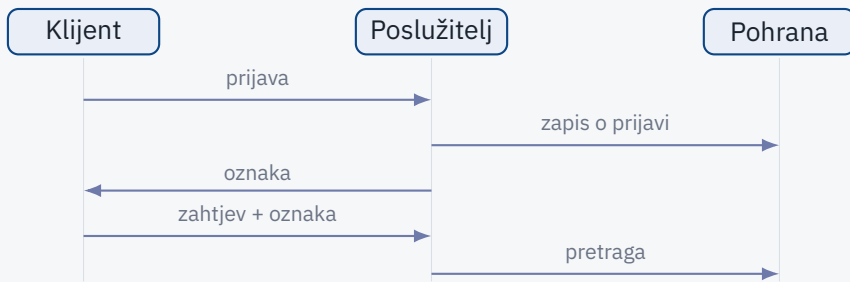
Prijava se dogodi jednom: korisnik pošalje nešto što samo on zna, a poslužitelj to provjeri. Slanje lozinke uz svaki sljedeći zahtjev nije rješenje, jer bi tražilo da je klijent trajno čuva i izlaže je u svakom zahtjevu.

Poslužitelj zato pri prijavi izdaje nešto drugo, što klijent prilaže svakom sljedećem zahtjevu. Dva su ustaljena oblika toga:

- **sjednica** (engl. *session*): nasumična oznaka koja sama ne znači ništa, a poslužitelj uz nju pamti tko je prijavljen
- **token**: potpisan zapis koji sam sadrži tko je korisnik, pa poslužitelj ne pamti ništa

Razlika je u tome tko čuva podatke o prijavi, a iz nje slijedi sve ostalo.

# Sjednica



Cookie: session=8f3c1a9e4b7d

Oznaka je samo ključ prema onome što poslužitelj pamti. Brisanjem zapisa prijava prestaje odmah.

# Token



Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Odgovara li potpis, sadržaju se vjeruje jer ga nitko bez ključa nije mogao sastaviti ni izmijeniti. Poslužitelj pritom ne zna koji su tokeni u optjecaju, pa ih ne može poništiti.

# Usporedba

|                            | Sjednica                 | Token                      |
|----------------------------|--------------------------|----------------------------|
| što zahtjev nosi           | nasumičnu oznaku         | potpisan zapis o korisniku |
| gdje su podatci            | na poslužitelju          | u samom zapisu             |
| provjera zahtjeva          | pretraga po oznaci       | provjera potpisa           |
| prekid pristupa            | brisanjem zapisa         | nije izravno moguć         |
| više primjeraka aplikacije | traži zajedničku pohranu | ne traži ništa zajedničko  |

Izbor se svodi na to što je skuplje: zajednička pohrana koju svaki primjerak aplikacije mora doseći, ili to što se korisniku pristup ne može oduzeti prije nego mu token istekne.

U ovom se projektu koristi token.

---

# Lozinke

Zašto se ne pohranjuju, hash i salt, izbor sporog algoritma, provjera pri prijavi.

# Zašto lozinka ne smije biti zapisana

Baza se može naći u tuđim rukama: krivo postavljenim pravom pristupa, ukradenom sigurnosnom kopijom, propustom u kodu.

Zapiše li se lozinka onakva kakva jest, napadač s kopijom baze ima svaki račun u sustavu. Šteta se time ne zaustavlja: dio korisnika istu lozinku koristi i drugdje, pa iz jedne baze slijede tuđi računi u sustavima koji s ovim nemaju veze.

Zahtjev je zato stroži od „nitko ne smije doći do baze”: iz sadržaja baze ne smije se moći doći do lozinke, čak ni kad je sadržaj poznat.

# Hash i salt

Hash je jednosmjerna funkcija: iz lozinke daje niz stalne duljine, a iz tog niza lozinka se ne može izračunati. Pri prijavi se ne uspoređuju lozinke nego hashevi.

Sam hash nije dovoljan. Ista lozinka uvijek daje isti niz, iz čega slijedi dvoje:

- unaprijed izračunata tablica parova lozinka i hash rješava cijelu bazu odjednom
- jednaki hashevi u tablici gotovo sigurno odaju korisnike s istom lozinkom

Zato se uz svaku lozinku stvara **salt**: nasumičan niz koji ulazi u izračun. Ista lozinka dvaju korisnika daje različit hash, unaprijed izračunate tablice postaju bezvrijedne, a salt se pohranjuje uz hash jer nije tajna.

# Zašto spor algoritam

Hash funkcije poput SHA-256 rade brzo, jer im je to svrha. Za lozinke je to nedostatak: napadač s kopijom baze pogađa lozinke koliko brzo ih može hashirati.

Algoritmi za lozinke zato imaju zadanu cijenu izračuna. Kod bcrypta je to **faktor rada**: svako povećanje za jedan udvostručuje posao.

- provjera pri prijavi traje desetak ili stotinjak milisekunda, što korisnik ne primijeti
- napad koji bi trajao sate produljuje se na godine

Vrijednost se bira tako da provjera ostane podnošljiva na poslužitelju; ono što je danas primjereno za nekoliko godina više nije, pa je ta vrijednost predmet povremene revizije.



# Provjera lozinke u kodu

```
# app/core/security.py
import bcrypt

def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())
```

`gensalt` stvara novi salt pri svakom pozivu i upisuje ga u sam niz, zajedno s oznakom algoritma i faktorom rada. Zasebni stupac za salt zato nije potreban.

`checkpw` iz zapisa iščita salt i faktor rada, ponovi izračun i usporedi rezultat. Usporedba traje jednako dugo bez obzira na to koliko se znakova podudara, pa iz njezina trajanja ništa ne slijedi.

# Ograničenje duljine kod bcrypta

Bcrypt u izračun uzima najviše **72 bajta** lozinke. Sve preko toga ne ulazi u hash.

Posljedica je tiha: dvije različite duge lozinke koje se podudaraju u prvih 72 bajta daju isti hash, pa obje prolaze provjeru. Pogreška se ne javlja ni pri postavljanju lozinke ni pri prijavi.

Zato se duljina lozinke ograničava u shemi na ulazu, ili se prije hashiranja primijeni sažimanje u niz stalne duljine. Algoritmi novijeg naraštaja to ograničenje nemaju.

---

# Implementacija tokena

Sadržaj i potpis, podatci u tokenu, izdavanje, vijek tokena i dvije vrste.

# Sadržaj tokena

JSON Web Token (JWT, RFC 7519) je niz od triju dijelova:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 <- zaglavlje  
.eyJzdWIiOiI3Iiwicm9sZSI6ImNsdWiiLCJleHAiOjE3OTI0ODMxNDB9 <- sadržaj  
.4pcPyMD09o1PSyXnrXCjTwXyr4BsezdI1AVTmud2fU4 <- potpis
```

Prva dva dijela su JSON zapisan u obliku `base64url`. Dekodirani glase:

```
{ "alg": "HS256", "typ": "JWT" } // zaglavlje  
{ "sub": "7", "role": "club", "exp": 1792483140 } // sadržaj
```

`base64url` nije šifriranje nego zapis binarnog sadržaja slovima i znamenkama: za dekodiranje ne treba ključ. To ne znači da je do tokena lako doći, jer ga na putu štiti TLS, a na klijentu mjesto na kojem je pohranjen. Znači da ga **onaj tko ga drži čita u cijelosti**. Iz toga slijedi pravilo: u token ne ulazi ništa što korisnik ne smije vidjeti.

# Potpis

Treći dio tokena poslužitelj računa iz prvih dvaju, pomoću tajnog ključa. Pri svakom zahtjevu isti izračun ponavlja nad primljenim zaglavljem i sadržajem, pa dobiveni niz uspoređuje s trećim dijelom koji je stigao uz njih.

- **Jamči:** sadržaj nije mijenjan nakon izdavanja i izdao ga je onaj tko poznaje ključ.
- **Ne jamči:** tajnost sadržaja ni to da token drži onaj kome je izdan.

Promijeni li napadač `role` u `admin`, izračun nad izmijenjenim sadržajem daje drukčiji niz od priloženoga i token pada na provjeri. Odgovarajući potpis ne može izračunati jer nema ključ.

Tko dođe do cijelog tokena, njime se služi kao da je vlasnik. Token je time izjednačen s lozinkom u pogledu čuvanja.

# Podatci u tokenu

| Oznaka  | Značenje                | Podrijetlo |
|---------|-------------------------|------------|
| sub     | na koga se token odnosi | standard   |
| exp     | trenutak isteka         | standard   |
| iss     | tko je token izdao      | standard   |
| type    | vrsta tokena            | aplikacija |
| role    | uloga korisnika         | aplikacija |
| club_id | klub korisnika          | aplikacija |

Uloga i klub u tokenu štede upit nad bazom pri svakoj provjeri prava. Cijena je zastarjelost: promijeni li organizator ulogu računu, token izdan prije toga i dalje nosi staru vrijednost do isteka.

Sadržaj se zato drži što manjim: ono što je potrebno za odluku, i ništa što se često mijenja.

# Izdavanje tokena

Nakon uspješne prijave poslužitelj sastavi sadržaj, doda trenutak isteka i potpiše cjelinu:

```
# app/core/jwt.py
def create_access_token(user_id: int, role: str, club_id: int | None) ->
    str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(user_id), "role": role, "club_id": club_id,
        "type": "access", "iss": settings.JWT_ISSUER,
        "exp": now + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
    }
    return jwt.encode(payload, settings.JWT_SECRET, algorithm=ALGORITHM)
```

Time je token izdan i poslužitelj o njemu više ništa ne pamti. Jedino što ograničava njegovu uporabu jest vrijednost upisana u exp.

## Koliko token smije trajati

Budući da se izdani token ne može poništiti, jedina je zaštita njegov vijek. Taj izbor povlači dvije suprotne posljedice:

| Vijek  | Ako token iscure        | Za korisnika          |
|--------|-------------------------|-----------------------|
| kratak | pristup prestaje brzo   | česta ponovna prijava |
| dug    | pristup traje do isteka | prijava jednom        |

Jednim tokenom bira se jedno ili drugo. Rješenje je izdati dva, s različitim vijekom i različitim putem: kratkotrajni koji putuje uz svaki zahtjev, i dugotrajni koji se šalje rijetko i služi samo za dobivanje novoga kratkotrajnog.



# Access token

Putuje uz svaki zahtjev i nosi sve što je potrebno za odluku o pravima:

```
{ "sub": "7", "role": "club", "club_id": 12,  
  "type": "access", "iss": "sit-unizd", "exp": 1792483140 }
```

Vrijedi kratko, u ovom projektu petnaest minuta. Kratak vijek je posljedica izloženosti: token koji se šalje stotinama puta na dan ima i najviše prilika da negdje ostane zabilježen, u zapisniku posrednika ili u povijesti preglednika. Istekne li, poslužitelj ga odbija s 401 i klijent zna da mu treba novi.

# Refresh token

Šalje se samo na rutu za obnovu i nosi najmanje što je potrebno:

```
{ "sub": "7", "type": "refresh",  
  "iss": "sit-unizd", "exp": 1793000000 }
```

Vrijedi dugo, u ovom projektu sedam dana. Dug vijek je podnošljiv jer token rijetko putuje, pa je i prilika da iscuri mnogo manja.

Oba tokena potpisana su istim ključem, pa bi bez oznake type refresh token prolazio i kao access token, i to sedam dana. Zato se vrsta provjerava na oba mjesta: ruta koja traži prijavljenog korisnika prihvaća samo access, a obnova samo refresh.

# Usporedba dvaju tokena

|            | <b>Access token</b> | <b>Refresh token</b>  |
|------------|---------------------|-----------------------|
| vijek      | 15 minuta           | 7 dana                |
| šalje se   | uz svaki zahtjev    | samo pri obnovi       |
| sadržaj    | uloga i klub        | samo oznaka korisnika |
| ako iscuri | kratak prozor       | pristup do isteka     |

Time se dvije suprotne potrebe razdvajaju: ono što je često izloženo kratko traje, a ono što dugo traje rijetko se šalje.

Korisnik od toga ne vidi ništa. Kad access token istekne, klijent refresh tokenom sam dobavi novi par i ponovi zahtjev.

---

## Prijava i provjera zahtjeva

Tijek prijave, ruta za prijavu, poruke o pogrešci, provjera pri svakom zahtjevu, obnova tokena.

# Tijek prijave

1. Klijent šalje adresu e-pošte i lozinku na rutu za prijavu.
2. Poslužitelj dohvaća korisnika i uspoređuje hash lozinke.
3. Ako se podudara, izdaje access i refresh token i vraća ih u odgovoru.
4. Klijent uz svaki sljedeći zahtjev šalje access token u zaglavlju `Authorization`.
5. Poslužitelj pri svakom zahtjevu provjerava token i iz njega dobiva korisnika.
6. Kad access token istekne, klijent refresh tokenom traži novi par.

Koraci od četvrtoga nadalje ponavljaju se u svakom zahtjevu. Prijava se pritom dogodila jednom, a njezin je jedini trag token koji klijent nosi sa sobom.

## Ruta za prijavu

```
# app/routers/auth.py
@router.post("/login", response_model=TokenResponse)
async def login(body: LoginRequest, db: AsyncSession = Depends(get_db)):
    user = await auth_service.authenticate_user(db, body.username, body.
        password)
    access, refresh = auth_service.create_tokens(user)
    return TokenResponse(access_token=access, refresh_token=refresh)

# app/services/auth_service.py
async def authenticate_user(db, username: str, password: str) -> User:
    user = await user_repo.get_by_username(db, username)
    if not user or not verify_password(password, user.password_hash):
        raise AppError("invalid_credentials", "...", 401)
    if not user.is_active:
        raise AppError("invalid_credentials", "...", 401)
    return user
```

Ruta prima i vraća, a odluka o tome je li prijava valjana pripada sloju services.

# Jedna poruka za tri slučaja

Nepostojeći korisnik, kriva lozinka i deaktiviran račun završavaju istim odgovorom i istom porukom.

Razlikovanje bi napadaču dalo podatak koji sam po sebi vrijedi: uz popis adresa e-pošte i rutu koja odaje postoji li račun, dobiva se popis korisnika sustava bez ijedne pogođene lozinke. Nad tim se popisom onda pogađa ciljano.

Isti razlog vrijedi i za rutu koja šalje poruku za promjenu lozinke: odgovor je jednak bez obzira na to postoji li adresa.

Iznimka su odgovori namijenjeni prijavljenom korisniku nad vlastitim podacima, gdje se ne odaje ništa što on već ne zna.

# Provjera tokena pri svakom zahtjevu

```
# app/core/deps.py
async def get_current_user(
    credentials: HTTPAuthorizationCredentials | None = Depends(
        _bearer_scheme),
    db: AsyncSession = Depends(get_db),
) -> User:
    if credentials is None:
        raise AppError("invalid_credentials", "...", 401)
    try:
        payload = decode_token(credentials.credentials)
    except JWTError:
        raise AppError("token_expired", "...", 401)
    if payload.get("type") != "access":
        raise AppError("invalid_credentials", "...", 401)
```

Nad tokenom radimo tri provjere: je li uopće poslan, može li se dekodirati i je li riječ o pristupnom tokenu. Nijedna ne dira bazu.



# Od tokena do korisnika

```
# app/core/deps.py, nastavak
user = await user_repo.get_by_id(db, int(payload["sub"]))
if not user or not user.is_active:
    raise AppError("invalid_credentials", "...", 401)
return user
```

Oznaka iz polja sub služi za dohvat zapisa, a zatim se provjerava je li račun aktivan.

Ruta koja traži prijavljenog korisnika navodi ovu ovisnost i dobiva zapis iz baze, a ne sadržaj tokena.

## Kako ruta dolazi do korisnika

Funkcija `get_current_user` stoji u `core`, a ruta je u `routers` ne poziva. Umjesto toga navodi je u svom potpisu:

```
# app/routers/auth.py
@router.get("/me", response_model=UserResponse)
async def get_me(current_user: User = Depends(get_current_user)):
    return current_user
```

`Depends` znači: prije poziva `get_me` pozovi `get_current_user` i ono što vrati predaj kao argument `current_user`. Takva se najava naziva **ovisnost** (engl. *dependency*), a poziva je framework. Iz toga slijedi:

- Provjera se izvodi samo u rutama koje je navedu. Ruta za prijavu i provjera dostupnosti je ne navode, pa im token ni ne treba.
- Iz potpisa funkcije vidi se traži li ruta prijavljenog korisnika.
- Korisnik u funkciju ulazi kao običan argument, pa ga test može zadati izravno, bez izdavanja tokena.

## Zašto se korisnik dohvaća iz baze

Token već nosi oznaku korisnika, ulogu i klub, pa se upit nad bazom čini suvišnim. Nije, jer token opisuje stanje u trenutku izdavanja.

U petnaest minuta koliko traje, korisnik može biti deaktiviran ili mu se može promijeniti uloga. Bez upita bi deaktiviran račun radio do isteka tokena, a to je upravo slučaj u kojem pristup treba prestati odmah.

Cijena je jedan upit po zahtjevu, po primarnom ključu. Aplikacije kojima je i to previše dio provjera prenose u token i prihvataju da promjene kasne; to je izbor, a ne propust.

# Obnova tokena

```
# app/services/auth_service.py
async def refresh_tokens(db, refresh_token: str) -> tuple[str, str]:
    try:
        payload = decode_token(refresh_token)
    except JWTError:
        raise AppError("invalid_credentials", "...", 401)
    if payload.get("type") != "refresh":
        raise AppError("invalid_credentials", "...", 401)
    user = await user_repo.get_by_id(db, int(payload["sub"]))
    if not user or not user.is_active:
        raise AppError("invalid_credentials", "...", 401)
    return create_tokens(user)
```

Obnova ponavlja iste provjere kao i svaki drugi zahtjev, uz jednu razliku u vrsti tokena. Vraća se novi par, pa i refresh token dobiva novi vijek.

---

## Što token ne rješava

Odjava i oduzimanje pristupa, pohrana tokena na klijentu, tajni ključ, pogađanje lozinki.

## Odjava i oduzimanje pristupa

Poslužitelj ne vodi popis izdanih tokena, pa ih ne može ni poništiti. Odjava na klijentu briše token iz pohrane, ali onaj tko je njegovu kopiju već preuzeo i dalje ga može upotrijebiti do isteka.

Isto vrijedi za promjenu lozinke i za deaktivaciju računa: bez dodatnog koraka stari token radi dalje.

| Mjera               | Kako djeluje          | Cijena             |
|---------------------|-----------------------|--------------------|
| kratak vijek tokena | skraćuje prozor       | češća obnova       |
| provjera u bazi     | prekida pristup odmah | upit po zahtjevu   |
| popis opozvanih     | prekida pristup odmah | zajednička pohrana |

U ovom projektu djeluju prve dvije: vijek od petnaest minuta i provjera polja `is_active` pri svakom zahtjevu.

## Pohrana tokena u pregledniku

|                         | <b>localStorage</b> | <b>Kolačić HttpOnly</b> |
|-------------------------|---------------------|-------------------------|
| čita ga JavaScript      | da                  | ne                      |
| šalje se sam uz zahtjev | ne                  | da                      |
| izložen napadu XSS      | da                  | ne                      |
| izložen napadu CSRF     | ne                  | da                      |

Obje su pohrane izložene, ali različitim napadima. Njih imenuje posljednja dva retka tablice.

# XSS i CSRF

**XSS** je izvođenje tuđeg JavaScripta u stranici: takav kod čita sve što i vlastiti, pa i `localStorage`.

**CSRF** je zahtjev koji tuđa stranica šalje u ime prijavljenog korisnika; preglednik kolačić prilaže sam, pa zahtjev prolazi kao vjerodostojan.

## Izbor, ne pravilo

Nijedna pohrana nije sigurna po sebi; bira se koji je napad vjerojatniji i čime se dopunjuje. U ovom projektu tokeni stoje u `localStorage`, uz kratak vijek pristupnog tokena.



# Tajni ključ

Cijela provjera počiva na jednoj vrijednosti. Tko je ima, izdaje tokene koje poslužitelj prihvaća, s bilo kojom ulogom.

- Ključ je nasumičan i dovoljno dug; `openssl rand -hex 32` daje primjerenu vrijednost.
- Ne stoji u kodu ni u repozitoriju nego u varijabli okoline, kao i ostala konfiguracija.
- Zadana vrijednost iz `.env.example` služi samo za razvoj i u produkciji se mijenja.

Promjena ključa poništava sve izdane tokene odjednom, jer nijedan potpis više ne odgovara. To je gruba mjera, ali jedina koja djeluje trenutačno bez dodatne pohrane.

## Pogađanje lozinki

Spor algoritam otežava pogađanje nad ukradenom bazom, ali ne sprječava pogađanje kroz samu rutu za prijavu. Napadač ondje ne treba hash: šalje pokušaje i čeka odgovor.

| Mjera                      | Sprječava                      |
|----------------------------|--------------------------------|
| spor algoritam             | pogađanje nad ukradenom bazom  |
| ograničenje broja pokušaja | pogađanje kroz rutu za prijavu |

Ograničenje se postavlja po adresi pošiljatelja ili po korisničkom računu, primjerice nekoliko pokušaja u minuti, nakon čega odgovor postaje 429. Mjera pritom ima i naličje: ograničenje po računu omogućuje da se tuđi račun namjerno zaključa nizom krivih pokušaja.

# Sažetak

- HTTP ne pamti pošiljatelja, pa identitet putuje u svakom zahtjevu posebno. Autentikacija odgovara tko šalje zahtjev, autorizacija smije li to učiniti.
- Lozinka se ne pohranjuje. Pohranjuje se hash sa saltom, dobiven algoritmom koji je namjerno spor.
- Token je potpisan, ali nije skriven: potpis jamči da sadržaj nije mijenjan i tko ga je izdao, a ne tajnost ni to da ga drži vlasnik.
- Kratak access token i dugotrajan refresh token razdvajaju izloženost od trajanja; oznaka vrste sprječava da drugi posluži umjesto prvoga.
- Poslužitelj ne vodi popis tokena, pa se pristup oduzima kratkim vijekom i provjerom stanja korisnika u bazi.